
🎯 Objective

The objective of this project is to evaluate students' understanding of advanced supervised learning regression techniques, with a strong focus on regularization, model generalization, cross-validation strategies, and tree-based regression algorithms. Students will learn how to control overfitting, select optimal models, and compare linear vs non-linear regressors using real-world data.

📑 Problem Statement

You are working as a **Machine Learning Engineer** at a real estate analytics company. The company already uses a basic regression model to predict **house prices**, but it suffers from **overfitting and unstable predictions** across different datasets.

Your task is to build a **robust regression pipeline** that:

- Applies **regularization techniques**
- Uses **proper cross-validation strategies**
- Compares **linear and non-linear regression models**
- Selects the **best-performing model** based on validation performance

The final solution must generalize well on unseen data.

🧠 Part A: Conceptual Foundation (Theory)

Answer briefly (2–4 lines each):

1. What is **Regularization** in Machine Learning? Why is it needed?
2. Difference between **Ridge Regression (L2)** and **Lasso Regression (L1)**.
3. What is **Cross-Validation** and why is it important?
4. Explain the following cross-validation techniques:
    - K-Fold Cross-Validation
    - Stratified K-Fold Cross-Validation
    - Leave-One-Out Cross-Validation (LOOCV)
    - Time Series Split
5. Why are **tree-based models** less sensitive to feature scaling?

🧪 Part B: Dataset Understanding & Preparation

You are provided with a **real estate dataset** containing:

- Property size and structural attributes
- Location-based numerical indicators
- Property age and renovation details
- Target variable: **House Price**

Dataset: Access from here

Tasks:

6. Identify **features and target variable**.
7. Perform a **train–test split**.
8. Apply **basic preprocessing** (scaling where required).

📉 Part C: Regularized Linear Models

9. Implement **Ridge Regression (L2)**.
10. Implement **Lasso Regression (L1)**.
11. Tune the **regularization parameter (α)** using **cross-validation**.
12. Compare Ridge and Lasso based on:
    - Training error
    - Validation error
    - Feature coefficient behavior

🔁 Part D: Cross-Validation Strategies

13. Apply and compare the following cross-validation techniques:
    - K-Fold Cross-Validation
    - Stratified K-Fold Cross-Validation *(by binning target values)*
    - Leave-One-Out Cross-Validation
    - Time Series Split *(using a time-related feature)*
14. Analyze how performance metrics vary across different CV strategies.

🌳 Part E: Tree-Based Regression Models

15. Implement **Decision Tree Regression**.
16. Control tree complexity using hyperparameters (max depth, min samples).
17. Implement **Random Forest Regression**.
18. Compare single-tree vs ensemble performance.

📐 Part F: Support Vector Regression

19. Implement **Support Vector Regression (SVR)** with:
    - Linear kernel
    - Polynomial or RBF kernel
20. Tune hyperparameters (C, γ, ε).
21. Compare SVR performance with linear and tree-based models.

📊 Part G: Model Comparison & Evaluation

22. Evaluate all models using appropriate regression metrics:
    - MSE, MAE, RMSE
    - R^2 Score
23. Compare:
    - Regularized Linear Models
    - Tree-Based Models

- Support Vector Regression

24. Identify signs of **overfitting or underfitting** in each model.

📄 Part H: Final Analysis & Reporting

25. Prepare a final report covering:
    - Best-performing model and justification
    - Impact of regularization
    - Role of cross-validation in model stability
    - Comparison of linear vs non-linear regressors
    - Business interpretation of results
26. Submit:
    - Source code / notebook
    - Evaluation tables and plots
    - Final conclusions

✅ Submission Guidelines

- Include practical implementation in a Jupyter Notebook and screenshots.
- Label all charts clearly and write short interpretations under each result.
- **GitHub Repository:**
    - Create a GitHub repository to host your project.
    - Upload your project files, including source code, and documentation to the repository.
    - Add a document (PDF) explaining theory concepts with definitions (if any).
    - Ensure that you provide a clear and descriptive README.md file.